

Assignment 20.3 Ai Assisted Coding

Htno:2303a51288

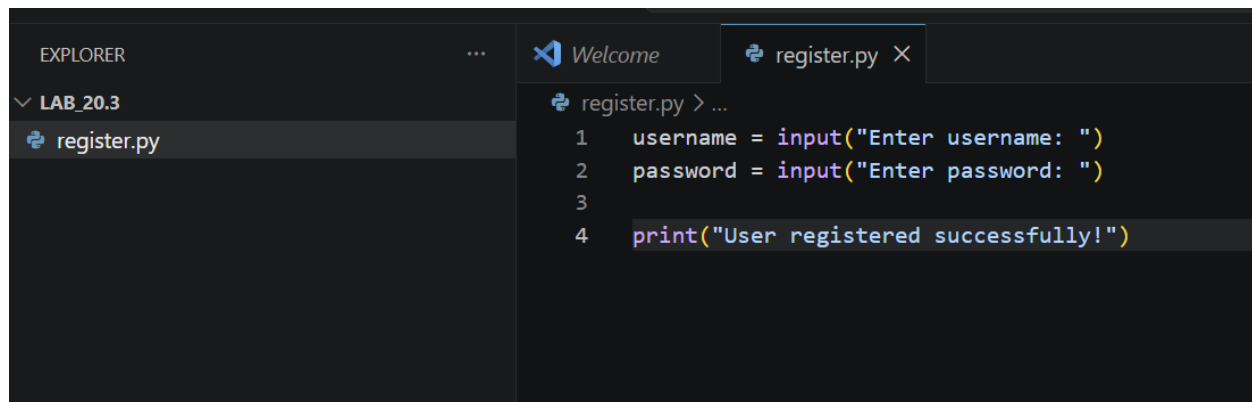
Btno:05

Task1: Password Strength & Validation Check

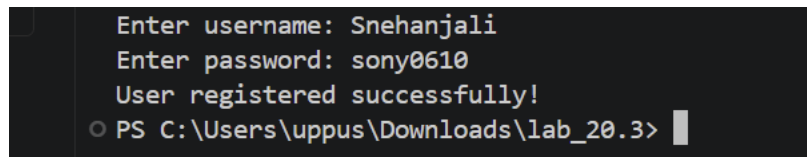
Prompt:Generate Python user registration program

Code:

AI-Generated Code:

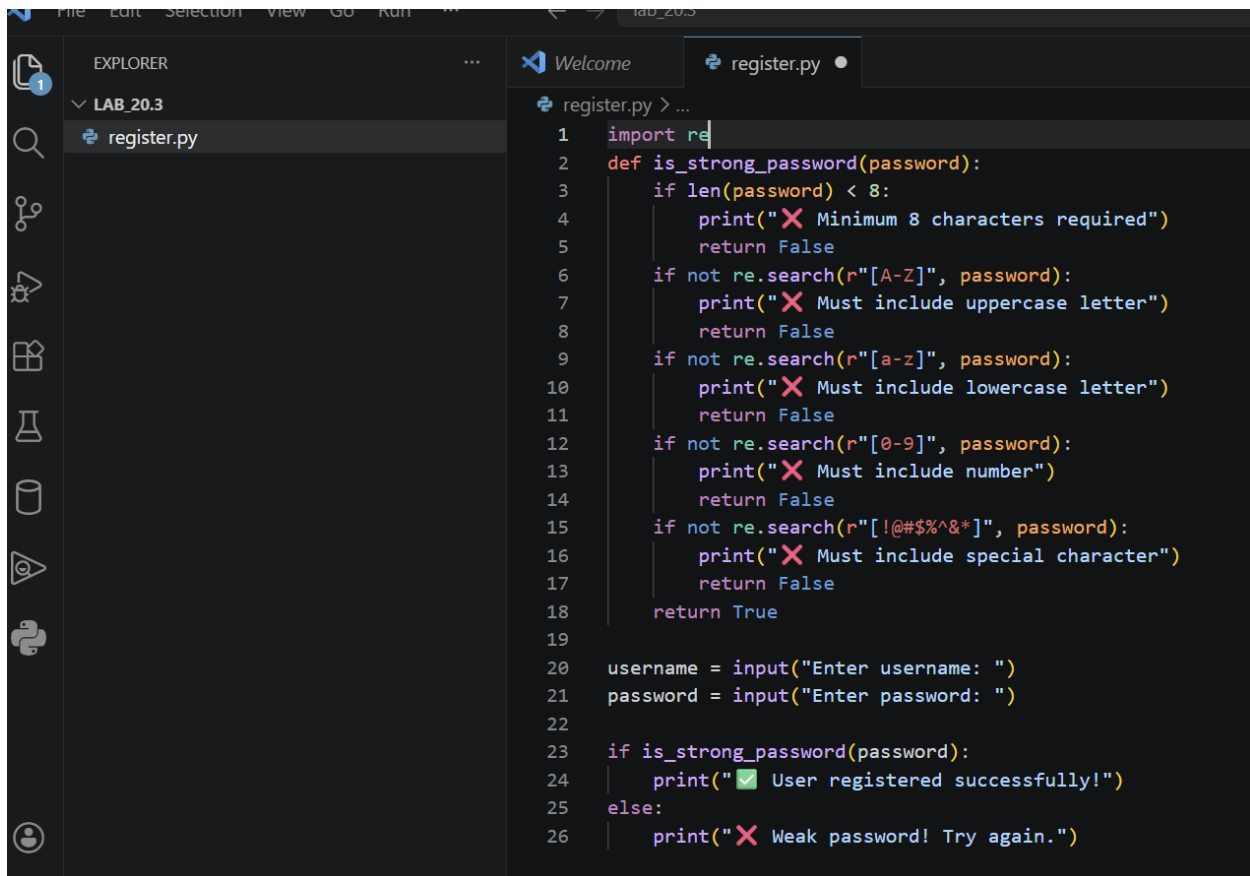
A screenshot of a code editor interface. On the left, the 'EXPLORER' pane shows a folder named 'LAB_20.3' containing a file 'register.py'. The main editor area shows the code for 'register.py' with four lines: 1. 'username = input("Enter username: ")', 2. 'password = input("Enter password: ")', 3. (empty line), and 4. 'print("User registered successfully!")'. The code is written in a dark-themed editor with syntax highlighting.

Output:

A screenshot of a terminal window. It shows the output of the Python program: 'Enter username: Snehanjali', 'Enter password: sony0610', and 'User registered successfully!'. Below the output, the command prompt shows 'PS C:\Users\uppus\Downloads\lab_20.3>' with a cursor.

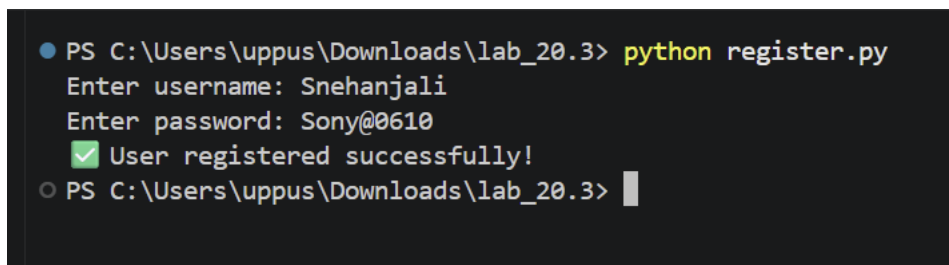
- The program accepts passwords without any validation checks.
- Weak passwords are allowed, making the system insecure.

Corrected Code:



```
1 import re
2 def is_strong_password(password):
3     if len(password) < 8:
4         print("❌ Minimum 8 characters required")
5         return False
6     if not re.search(r"[A-Z]", password):
7         print("❌ Must include uppercase letter")
8         return False
9     if not re.search(r"[a-z]", password):
10        print("❌ Must include lowercase letter")
11        return False
12    if not re.search(r"[0-9]", password):
13        print("❌ Must include number")
14        return False
15    if not re.search(r"[!@#$$%^&*]", password):
16        print("❌ Must include special character")
17        return False
18    return True
19
20 username = input("Enter username: ")
21 password = input("Enter password: ")
22
23 if is_strong_password(password):
24     print("✅ User registered successfully!")
25 else:
26     print("❌ Weak password! Try again.")
```

Output:



```
PS C:\Users\uppus\Downloads\lab_20.3> python register.py
Enter username: Snehanjali
Enter password: Sony@0610
✅ User registered successfully!
PS C:\Users\uppus\Downloads\lab_20.3>
```

Explanation:

The initial program does not check the strength of the password. It accepts any password entered by the user, including weak ones. This makes the system insecure and vulnerable to attacks.

Task2: Insecure File Upload Handling

Prompt:

Generate a Flask-based file upload program that allows users to upload files.

Code: Ai Generated Code

```
1 from flask import Flask, request, render_template_string
2
3 app = Flask(__name__)
4
5 html = '''
6 <form method="POST" action="/upload" enctype="multipart/form-data">
7   <input type="file" name="file">
8   <input type="submit">
9 </form>
10 '''
11
12 @app.route('/')
13 def home():
14     return html
15
16 @app.route('/upload', methods=['POST'])
17 def upload_file():
18     file = request.files.get('file')
19     if file:
20         file.save(file.filename)
21         return "File uploaded successfully!"
22     return "No file!"
23
```

PROBLEMS OUTPUT TERMINAL PORTS

DEBUG CONSOLE TERMINAL

PS C:\Users\uppus\Downloads\lab_20.3> python upload.py
127.0.0.1 - - [01/Apr/2026 22:06:41] "POST /upload HTTP/1.1" 200 -

Output:



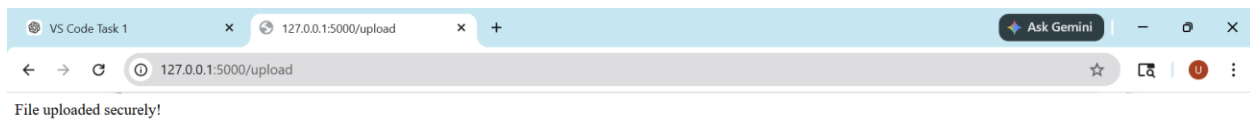
File uploaded successfully!

- The program allows uploading any type of file
- No validation of file extension

Corrected code:

```
1 from flask import Flask, request, render_template_string
2 import os
3
4 app = Flask(__name__)
5
6 UPLOAD_FOLDER = "uploads"
7 ALLOWED_EXTENSIONS = {"txt", "png", "jpg", "pdf"}
8
9 if not os.path.exists(UPLOAD_FOLDER):
10     os.makedirs(UPLOAD_FOLDER)
11
12 def allowed_file(filename):
13     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
14
15 html = '''
16 <form method="POST" action="/upload" enctype="multipart/form-data">
17   <input type="file" name="file">
18   <input type="submit">
19 </form>
20 '''
21
22 @app.route('/')
23 def index():
24     return render_template_string(html)
25
26 @app.route('/upload', methods=['POST'])
27 def upload():
28     if 'file' not in request.files:
29         return "No file part"
30     file = request.files['file']
31     if file.filename == '':
32         return "No selected file"
33     if file and allowed_file(file.filename):
34         filename = secure_filename(file.filename)
35         file.save(os.path.join(UPLOAD_FOLDER, filename))
36         return "File uploaded successfully!"
37     return "Invalid file type"
```

Output:



Explanation:

- The initial code allows uploading any file without checking its type, making it insecure.
- Malicious files like .exe can be uploaded, which may harm the system.

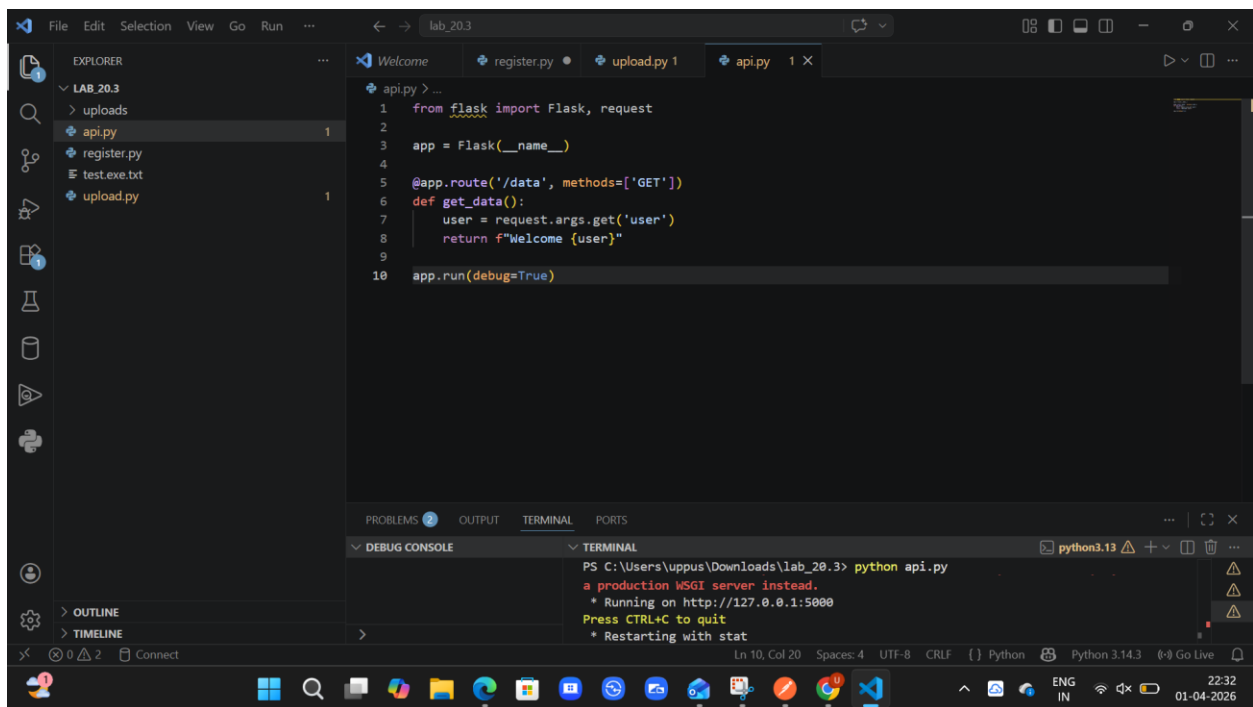
- The improved code restricts file types and allows only safe formats like .txt, .png, .jpg, and .pdf.

Task3: API Security Review

Prompt:

Generate a Python Flask API endpoint for user data

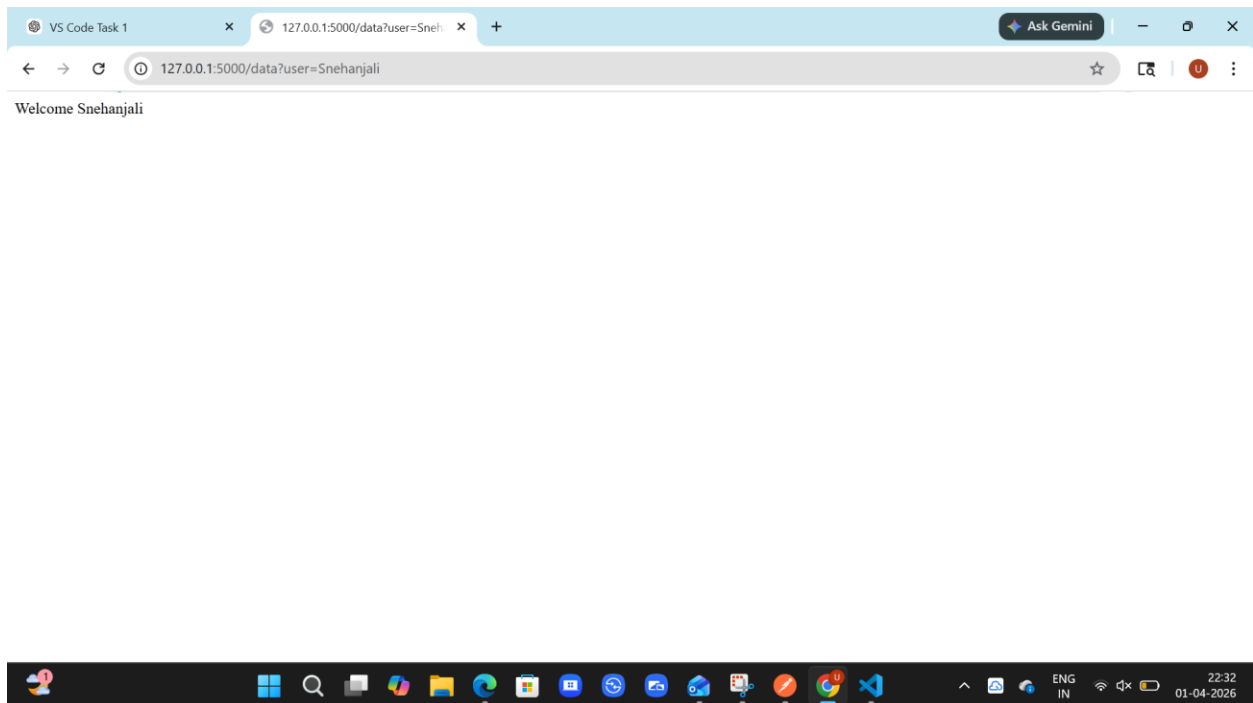
Code:



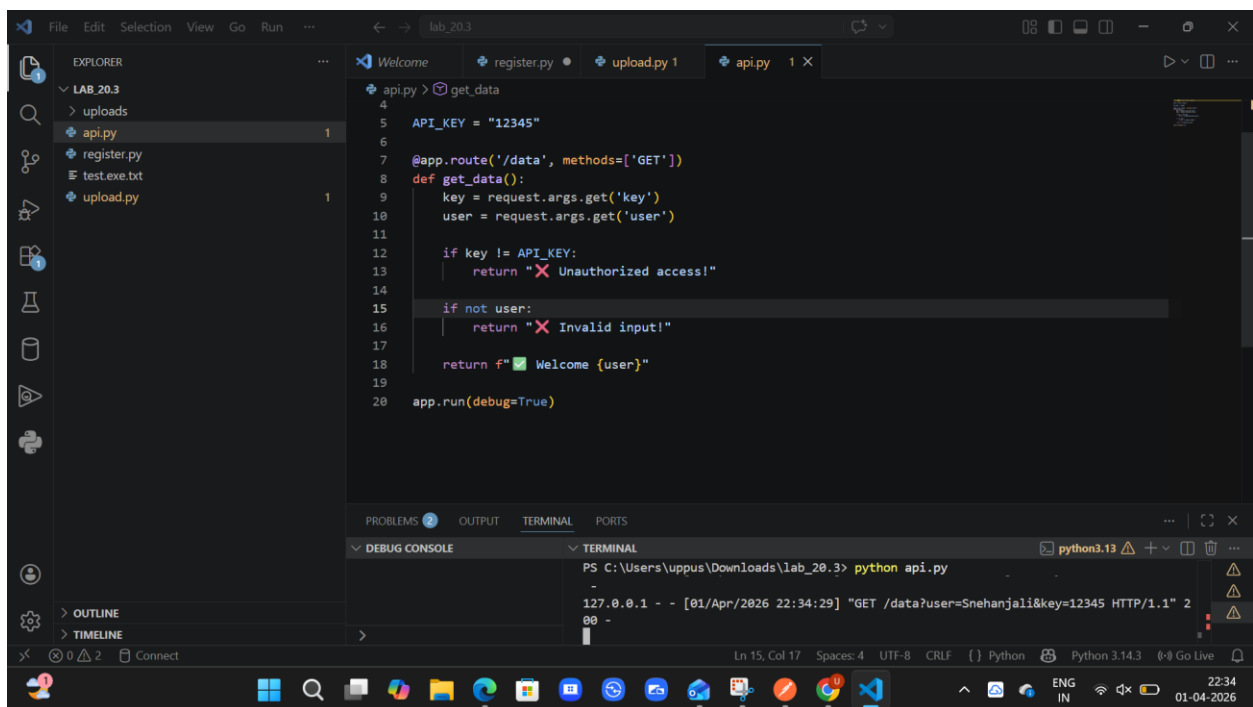
```
File Edit Selection View Go Run ... lab_20.3
EXPLORER
  LAB_20.3
    uploads
      api.py
      register.py
      test.exe.txt
      upload.py
  PROBLEMS
  OUTPUT
  TERMINAL
  PORTS
  DEBUG CONSOLE
  TERMINAL
    PS C:\Users\uppus\Downloads\lab_20.3> python api.py
    a production WSGI server instead.
    * Running on http://127.0.0.1:5000
    Press CTRL+C to quit
    * Restarting with stat
```

```
api.py > ...
1  from flask import Flask, request
2
3  app = Flask(__name__)
4
5  @app.route('/data', methods=['GET'])
6  def get_data():
7      user = request.args.get('user')
8      return f"Welcome {user}"
9
10 app.run(debug=True)
```

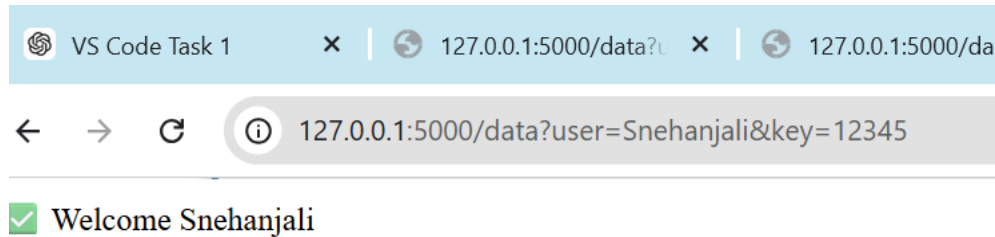
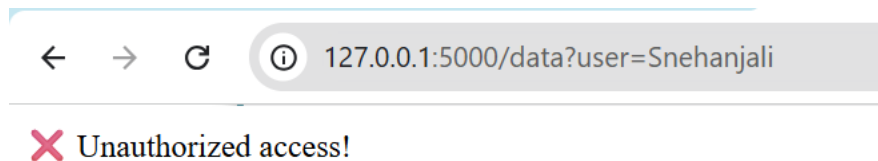
Output:



Corrected code:



Output:



Explanation:

- The initial API does not have authentication, allowing anyone to access it
- It does not validate user input, making it insecure
- The improved API uses an API key and input validation for secure access

Task 4: Cross-Site Scripting (XSS) Check

Prompt: Generate a feedback form using HTML and JavaScript

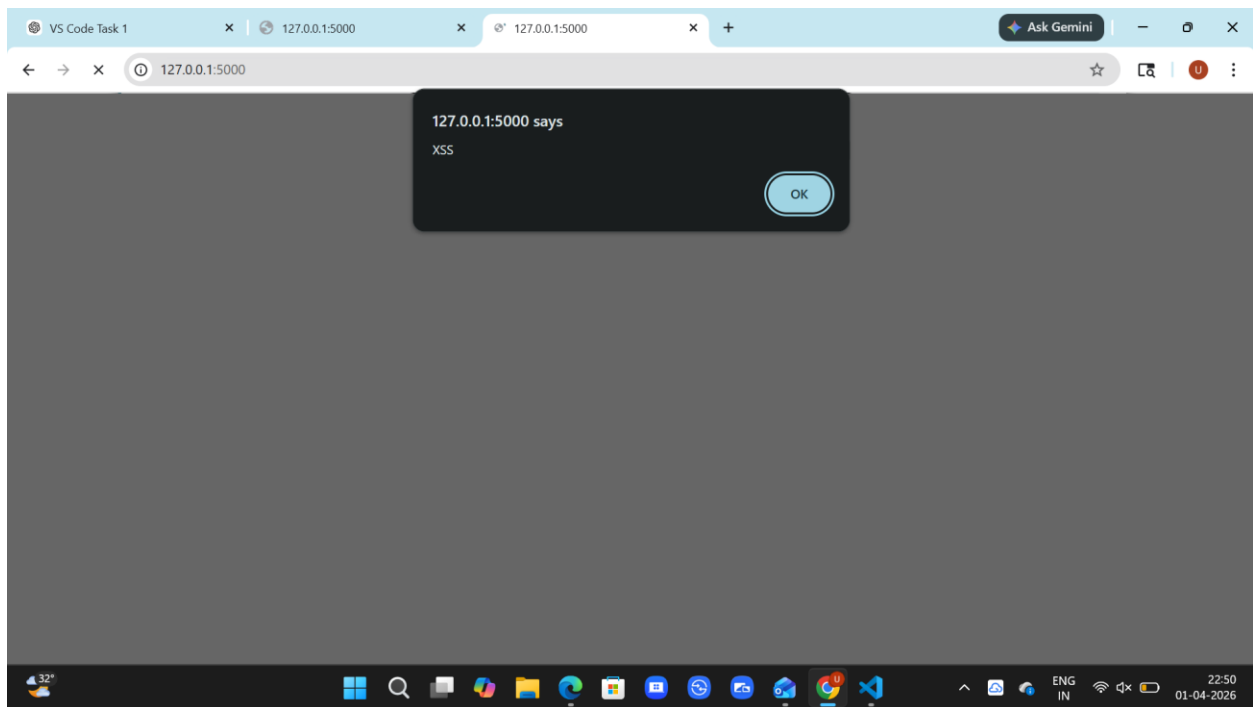
Code:

```
1 #!/usr/bin/env python
2
3 app = Flask(__name__)
4
5 @app.route('/', methods=['GET', 'POST'])
6 def form():
7     if request.method == 'POST':
8         name = request.form['name']
9         return f"Hello {name}"
10
11     return '''
12     <form method="POST">
13         <input type="text" name="name">
14         <input type="submit">
15     </form>
16     '''
17
18 app.run(debug=True)
```

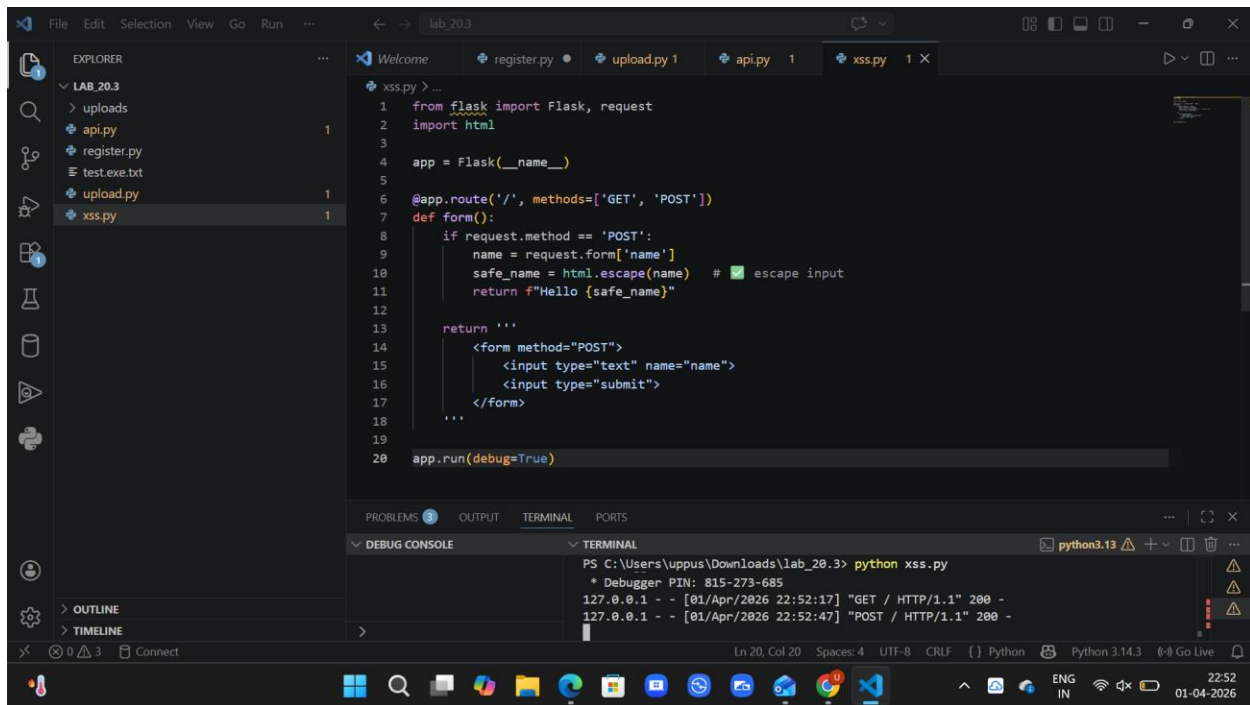
TERMINAL

```
PS C:\Users\uppus\Downloads\lab_20.3> python xss.py
127.0.0.1 - - [01/Apr/2026 22:47:28] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:48:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:49:08] "POST / HTTP/1.1" 200 -
```

Output:



Corrected Code:

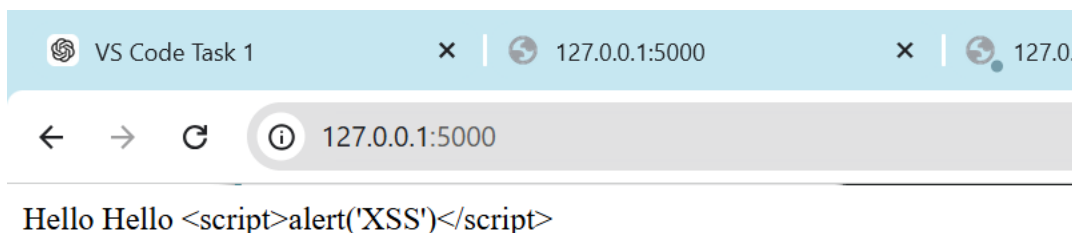


```
1 from flask import Flask, request
2 import html
3
4 app = Flask(__name__)
5
6 @app.route('/', methods=['GET', 'POST'])
7 def form():
8     if request.method == 'POST':
9         name = request.form['name']
10        safe_name = html.escape(name) # escape input
11        return f"Hello {safe_name}"
12
13    return '''
14    <form method="POST">
15        <input type="text" name="name">
16        <input type="submit">
17    </form>
18    '''
19
20 app.run(debug=True)
```

TERMINAL

```
PS C:\Users\uppus\Downloads\lab_20.3> python xss.py
* Debugger PIN: 815-273-685
127.0.0.1 - - [01/Apr/2026 22:52:17] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:52:47] "POST / HTTP/1.1" 200 -
```

Output:



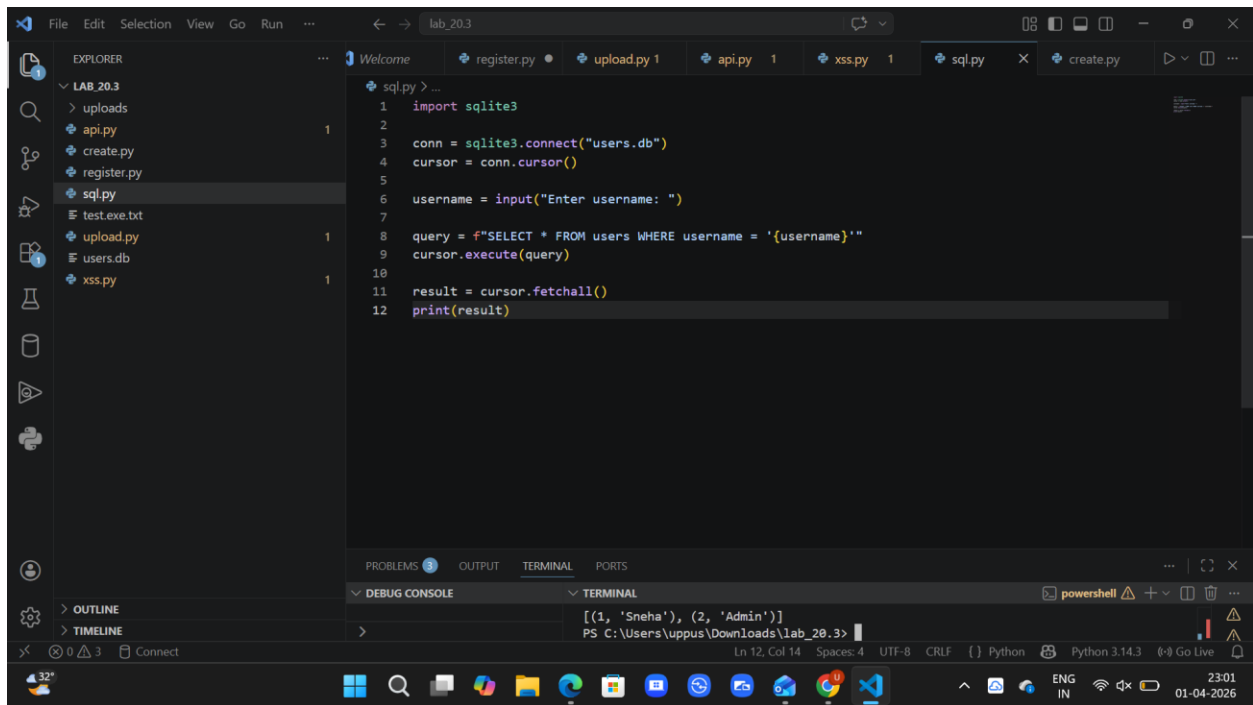
Explanation:

- The initial code displays user input directly without any validation, making it vulnerable to XSS attacks.
- Malicious scripts like `<script>alert('XSS')</script>` can execute in the browser.
- The secure code uses input escaping to prevent script execution and ensures safe output.

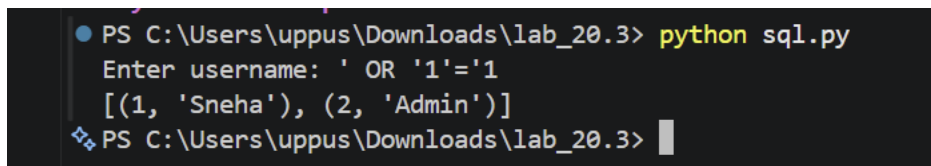
Task 5: SQL Injection Prevention

Prompt: Generate a Python script using SQLite to fetch user details from a database.

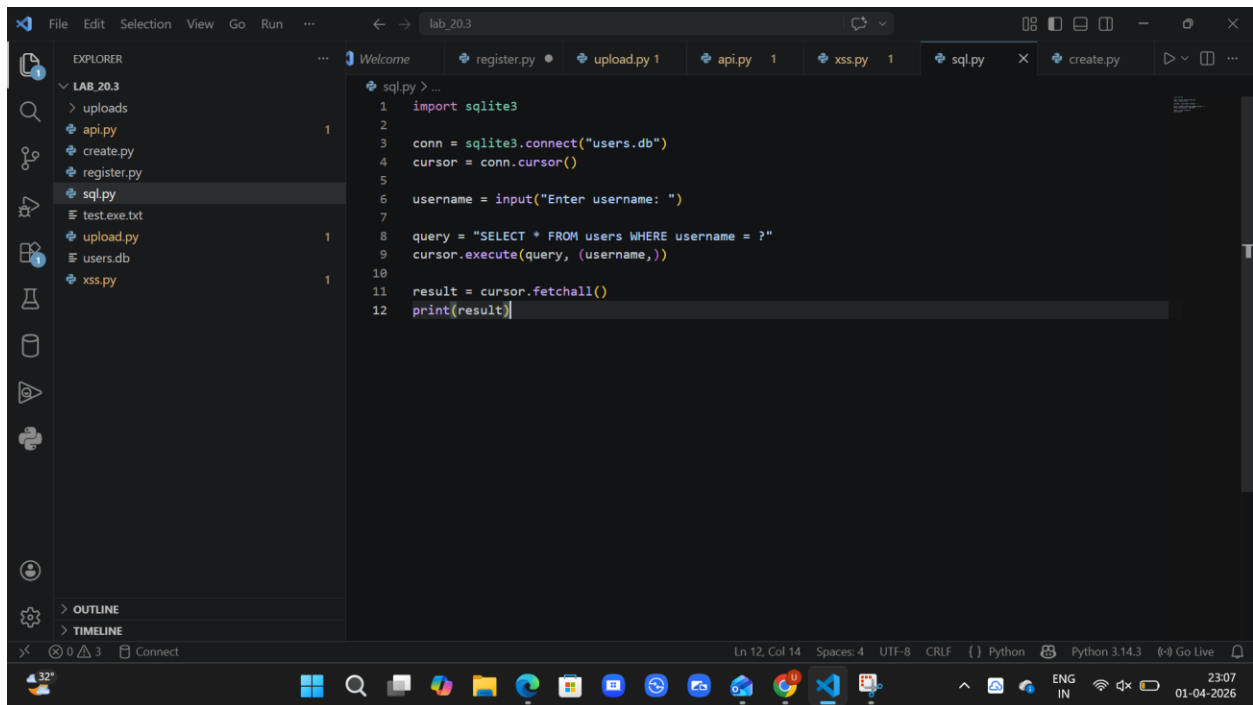
Code:



Output:



Corrected code:



Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[]
PS C:\Users\uppus\Downloads\lab_20.3> 
```

Explanation:

- The secure code uses parameterized queries (?) instead of string concatenation.
- User input is treated as data, not as part of the SQL query.
- This prevents SQL Injection and ensures safe database operations.

